UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

--------***--------

REPORT LABWORK 3

MACHINE LEARNING AND DATA MINING II

CLASS: 1

	STUDENT NAME	STUDENT ID
1	Nguyễn Quang Hưng	22BI13184
2	Vũ Tuấn Hải	22BI13149

Ha Noi, March 5th 2025

For this labwork, we select a total of 4 datasets available in the *scikit-learn* library. In the first part, the two datasets are Iris and Breast Cancer. In the second part, we select two datasets, namely the Wine dataset and Digits dataset.

K-nearest neighbor classification

I. Dataset

1. Dataset overview

- The *Iris dataset* contains 150 samples, 4 features, and 3 labels about different types of iris flowers named Setosa, Versicolour, and Virginica.
- The *Breast Cancer dataset* is a dataset concerning the diagnosis of breast cancer. It has 569 samples, 30 features (from the characteristics of each cell) to identify tumors based on 2 labels (malignant or benign).

2. Dataset processing

We split each dataset to train and test in a 7/3 ratio (70% for training and 30% for testing) to evaluate the model's generalization performance.

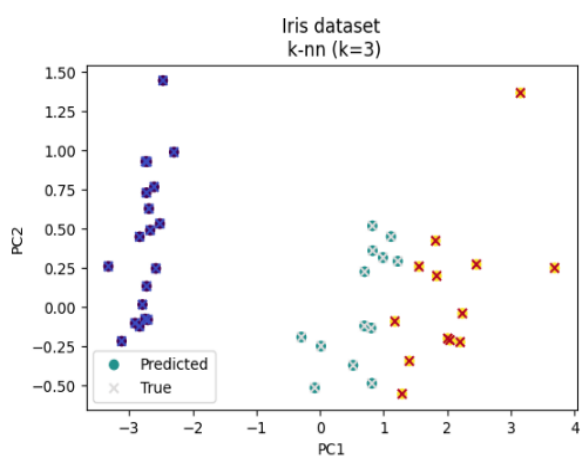
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

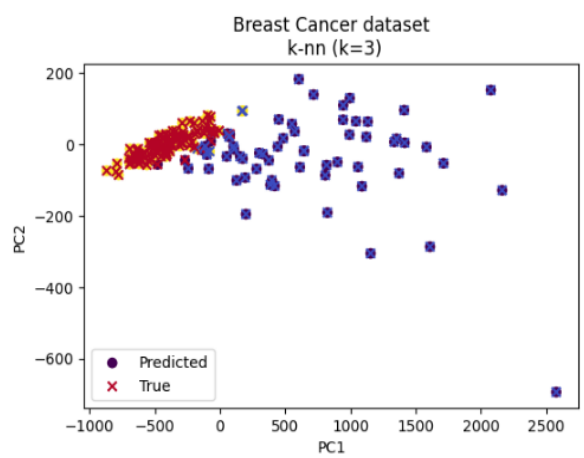
II. Methodology processing

1. Run k-NN and calculate the classification error

We choose k = 3, the classification error is calculated using the below formula:

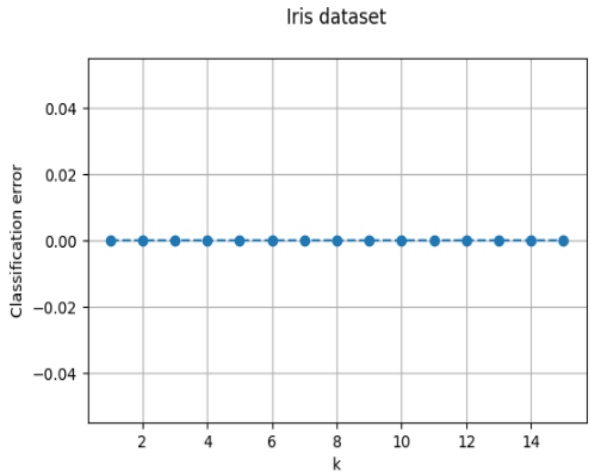
$$\boldsymbol{Error = 1 - accuracy}$$

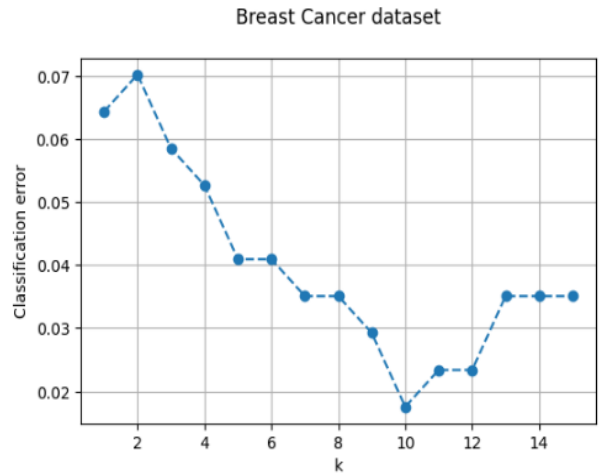




2. Vary the value of k

We start to initiate k-nn with the value from 1 to 15. The classification error for each k is plotted to visualize the relationship between k and model performance.



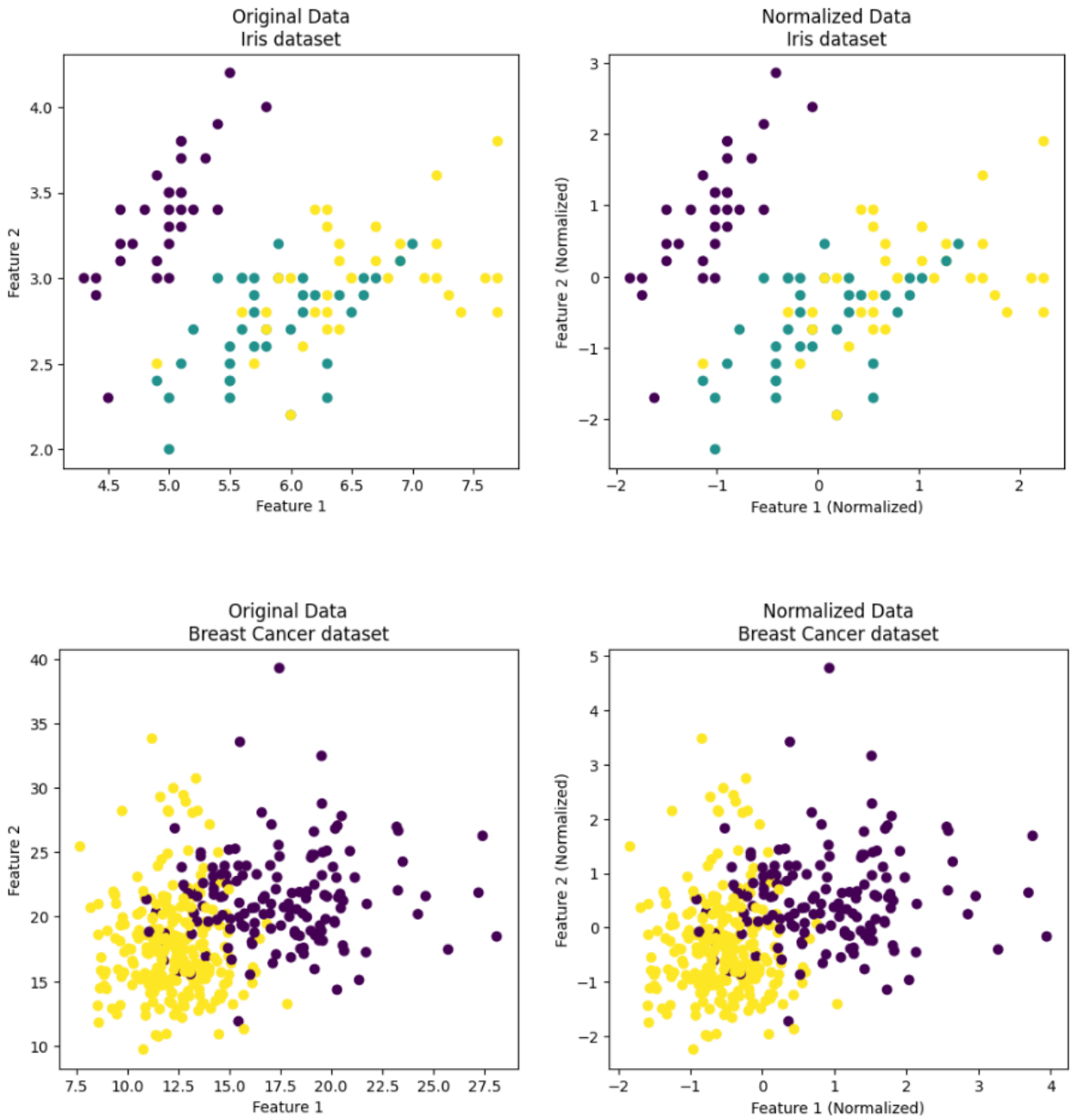


Look at the result, value k has a great effect on the model's performance

- With a very small k (like k = 1), the model is easily interfered (overfitting).
- With a very large k (like k = 15), local information can be lost, leading to underfitting.

3. Normalize data

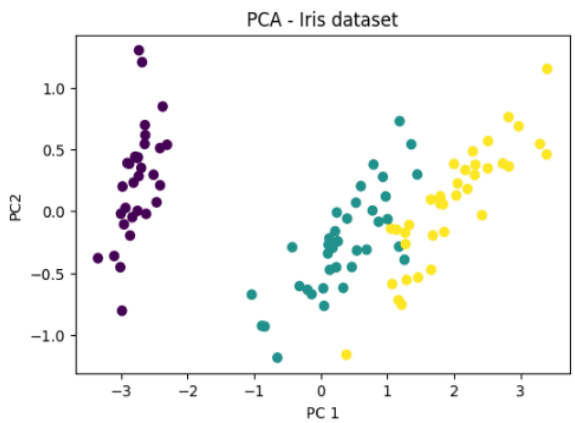
In this process, we use *StandardScaler* to transform data that have a mean of 0 and a standard deviation of 1, then we apply k-nn with k = 3.

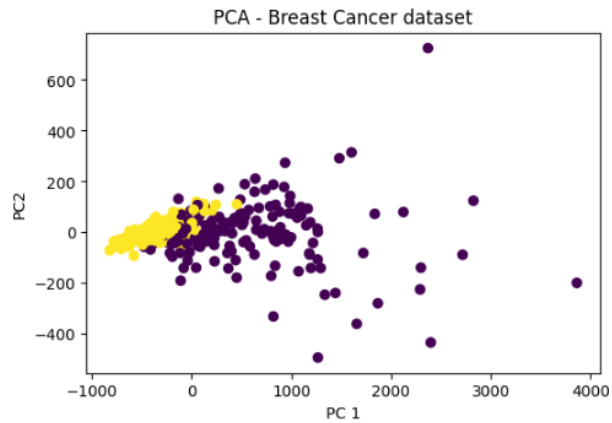


After normalizing, the performance of k-NN has improved because the distances between points are calculated much more precisely when all features stay at the same scale.

4. PCA and SVD

- Principal component analysis (PCA) reduces the feature space to 2 principal components, capturing most of the data's variance, which are then used to train the model.



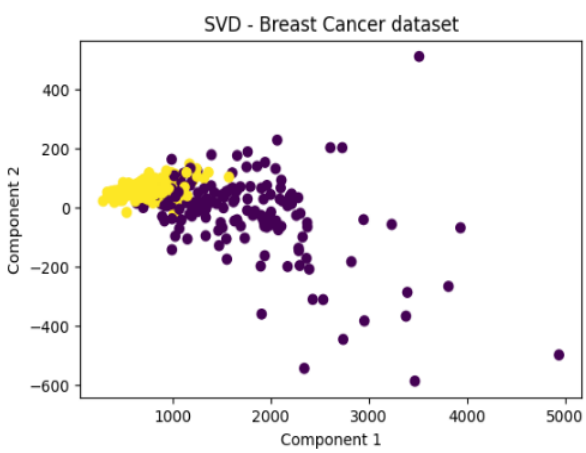


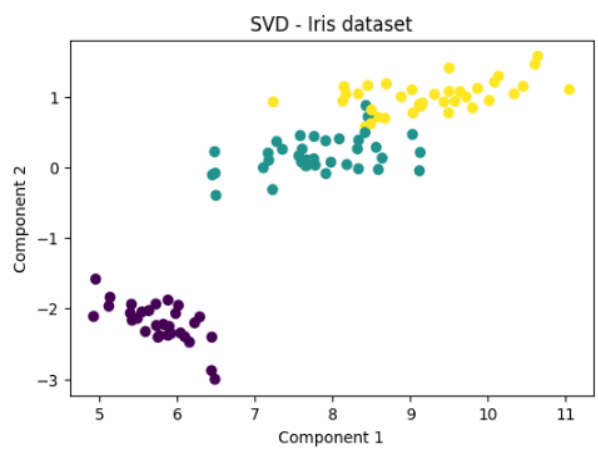
- Singular value decomposition (SVD) is similar to PCA, used for reducing the dimensionality of the data. SVD is often used in discrete data to decompose a matrix A, which makes it easier for analysis.

$$\boldsymbol{A = UMV^T}$$

where U: general preferences
M: how much each factor rating matters
V^T: how similar products are to each other

We use *TruncatedSVD* to reduce the data into 2 dimensions



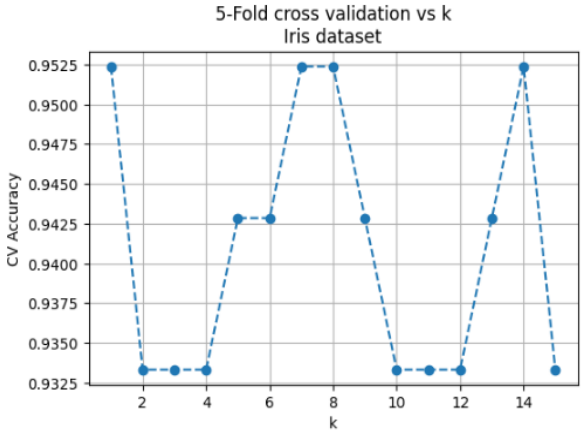


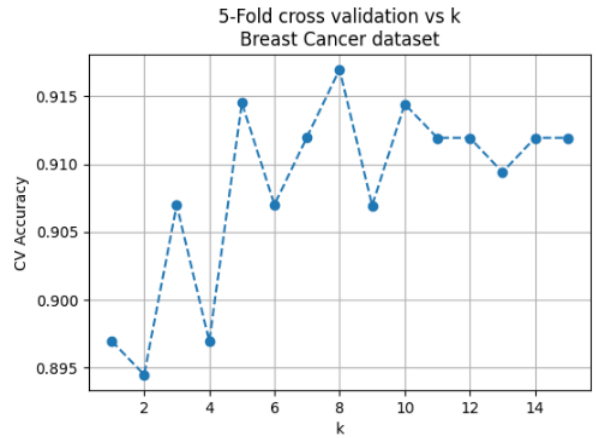
Reducing dimensionality may result in information loss, but if the data has more redundant features, performance can still improve. The output depends on the amount of information retained during the reduction process.

5. K-Fold cross validation

- Cross-validation is a statistical method that divides the available data into multiple folds. This process is repeated multiple times, each time using a different fold as the validation set.

- In this process, we choose k = 5 (based on the highest average accuracy across folds). This helps in preventing overfitting.





6. Apply leave-one-out and calculate the error of classification

Leave-one-out cross validation (LOOCV) is a type of cross validation approach, applied with k-nn (with k = 3). This method computes the average classification error when each sample is left out once as the test set.

$$CV_{(n)} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - y_i}{1 - h_i}\right)^2$$

```
LOOCV classification error with k=3: 0.0400

LOOCV classification error with k=3: 0.0738
```

SVM classifier

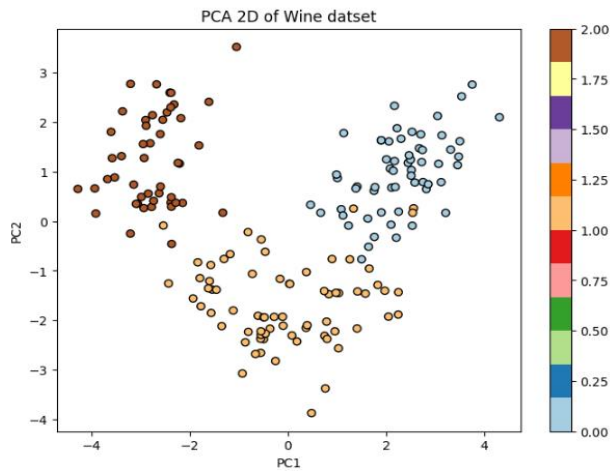
I. Dataset

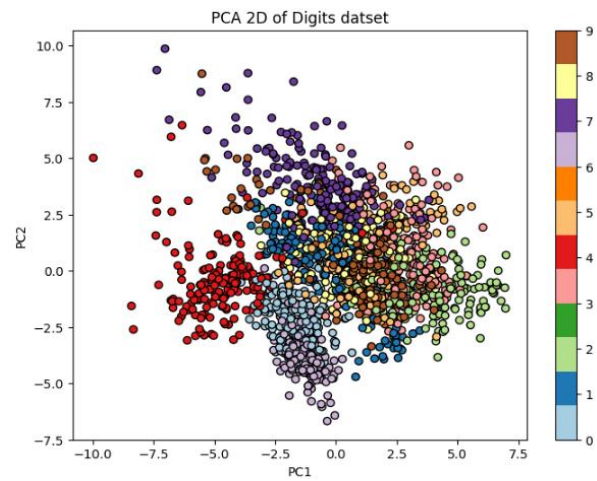
1. Dataset overview

- The *Wine dataset* includes 178 samples and 13 features, and the labels are different types of wines.
- The *Digits dataset* contains 1797 samples (8x8 images flattened to 64 features) representing 10 labels, which are digits from 0 to 9.

2. Data analysis

- In the Wine dataset, using PCA to reduce to 2 dimensions shows that the labels tend to form clusters with some overlap, so the data distribution is *linearly separable*.
- In the Digits dataset, PCA projection shows that the labels are not well separated by straight lines, so the data distribution is *non-linearly separable*.





II. Apply SVM

1. Set up the SVM parameters

- Wine dataset:
  - Kernel: We choose a linear kernel here because of the near linear separability shown in PCA visualization.
  - Parameter C: We set C = 1, which is a standard to handle misclassification.

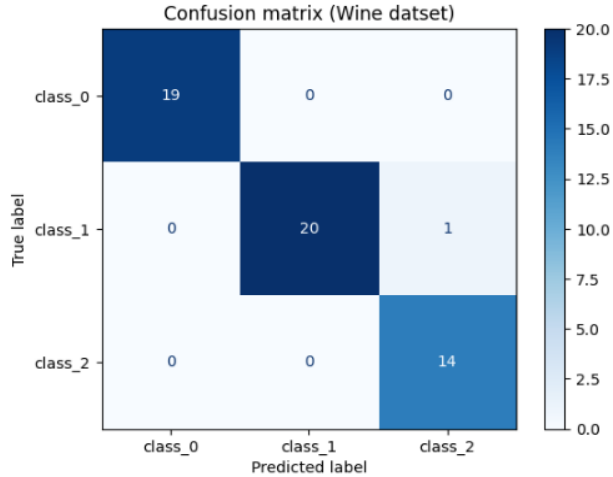
```
run_svm_experiment(wine, "Wine", kernel='linear', C=1)
```

- Digits dataset:
  - Kernel: Radial Basic Function (RBF) kernel was chosen to capture the non-linear relationships in the data.
  - Parameter C: Set C to 10
  - Gamma: Set to 'scale' to automatically compute the kernel coefficient based on the input data.

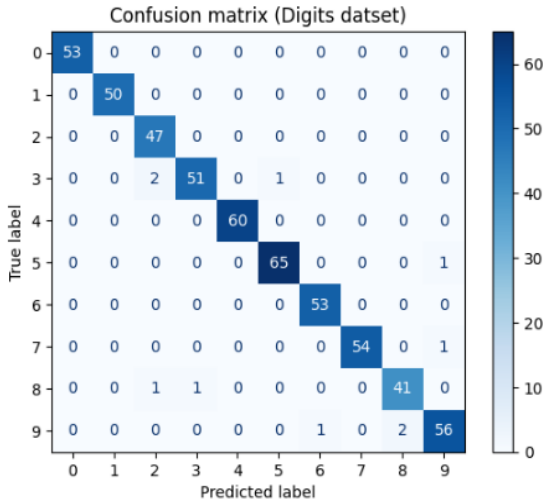
```
run_svm_experiment(digits, "Digits", kernel='rbf', C=10, gamma='scale')
```

2. SVMs performance

- The SVM (using linear kernel) achieved a high test accuracy on the Wine dataset. The confusion matrix shows that most classes were correctly classified.



- In the Digits dataset, the more complex nature is well-handled by the SVM (with RBF kernel), which effectively captures non-linear boundaries.



3. SVM handles multi-class datasets

When SVM meets multi-class datasets, sklearn in Python will apply two strategies:

- ***One vs One (ovo)***: For each pair of classes, a binary classifier is trained. During prediction, each classifier votes and the class with the most votes is selected. We also use this approach for both datasets.
- ***One vs Rest (ovr)***: A binary classifier is trained for each class. The final prediction is based on the classifier with the highest confidence score.